Gestion des transactions concurrentes – FestiBed

Projet intégrateur M1 MIAGE – Groupe 3 – 22/10/2026 – Oracle · Kotlin Spring · JPA

1. Le problème

FestiBed vend deux ressources **limitées** que plusieurs festivaliers peuvent vouloir en même temps :

- **les Pass** d'un festival, limités par `quotaPass` ;
- **les nuitées** d'un hébergement : une chambre ne peut être occupée qu'une fois par nuit.

Le risque est la **survente** : deux festivaliers voient la même dernière place, paient tous les deux, et l'un d'eux n'a rien. Le sujet impose de garantir les propriétés ACID et de tenir la montée en charge (beaucoup de demandes simultanées sur un festival populaire).

2. Principe retenu (validé à l'audit du 21/10)

> **Rien n'est écrit en base avant le paiement.** Le panier (les « sélections ») vit dans l'IHM, 15 minutes au plus. La réservation, les lignes, les Pass et l'occupation des nuitées sont créés **en une seule transaction courte**, au moment de la validation payée.

- Aucun verrou n'est tenu pendant que le festivalier réfléchit (contrôle **optimiste**) : c'est indispensable pour monter en charge.
- Les disponibilités affichées pendant la recherche sont **indicatives**. La seule vérification qui fait foi est celle faite **dans** la transaction finale (le prof : « la concurrence, c'est quand il paye »).
- Il est inutile de revérifier à chaque étape intermédiaire du parcours.

3. Ce que le MCD apporte déjà (points forts)

Élément du modèle	Pourquoi il protège
Nuitee identifiée par (hébergement, date), par composition	L'identifiant naturel interdit d'avoir deux fois la même nuit pour une chambre. C'est la clé primaire qui garantit l'unicité, pas le code applicatif.
Occupation Nuitee 0,1 — Ligne_reservation	Une nuitée est occupée par **au plus une** ligne. Réserver revient à faire passer `idLigneReservation` de NULL à une valeur, et un seul UPDATE peut réussir.
Pass rattaché au Festival et à la Ligne_reservation	Chaque Pass est une place d'un festival précis. On compte les places vendues par festival sans ambiguïté.
Reservation → Ligne_reservation → Pass / Nuitee	Tout ce qui est créé à la validation dépend de la réservation : une seule transaction englobe l'ensemble.
Paiement avec `statutPaiement` (dont REMBOURSE)	Permet de tracer le cas « payé mais plus disponible » et de rembourser.

4. Points sensibles

> **1. Le quota de Pass est un « point chaud ».** Pour un festival très demandé, toutes les transactions touchent la même ligne Festival. Compter les Pass vendus (`COUNT(*)`) sous verrou est lent, et sans verrou c'est faux.
> *Solution :* un compteur `nbPassVendus` dans Festival et un UPDATE conditionnel atomique (§5). Le verrou ne dure alors que quelques millisecondes.

2. Paiement accepté, mais transaction refusée. Le paiement se fait avant la transaction : entre les deux, un autre festivalier peut prendre la place.
Solution : rembourser la partie non servie (`statutPaiement = REMBOURSE`). Mieux encore, si la passerelle de paiement le permet : **autoriser** le montant, faire la transaction, puis **encaisser** le montant réellement servi.

3. Interblocages (deadlocks). Une réservation peut verrouiller plusieurs nuitées et plusieurs festivals. Deux transactions qui les prennent dans un ordre différent se bloquent mutuellement.
Solution : toujours verrouiller dans le même ordre : festivals par `idFestival`, puis nuitées par (établissement, hébergement, date).

4. Échec partiel d'une réservation. Une réservation contient plusieurs sélections. Selon l'audit, seule la ligne en échec « saute », pas toute la réservation.
Solution : un `SAVEPOINT` par ligne. En cas d'échec, `ROLLBACK TO SAVEPOINT`, la ligne est retirée et le montant ajusté.

5. Plusieurs hébergements pour couvrir les Pass (ex. 100 Pass, des chambres de 2 à 4 places). La validation doit occuper **toutes** les chambres choisies. Si l'une a été prise entre-temps, on applique la règle du BPMN (prendre ce qui reste, ou retirer la sélection).

6. Unicité du code Pass. Il est généré dans la transaction (séquence Oracle ou UUID) et protégé par une contrainte UNIQUE : pas de collision possible.

7. Purge du panier (15 min). Le panier n'est pas en base : sa purge n'a aucun impact transactionnel. C'est un point fort de notre choix.

5. Mise en œuvre dans notre modèle

Isolation Oracle par défaut (**READ COMMITTED**) et verrous de ligne. Toute la validation tient dans **une** méthode `@Transactional` côté serveur.

```
BEGIN  -- validation d'un panier payé
  INSERT INTO Reservation (idReservation, dateReservation, idFestivalier) VALUES (...);
  INSERT INTO Paiement (...) VALUES (..., 'ACCEPTE');

  POUR CHAQUE sélection, triée par idFestival puis par hébergement :
    SAVEPOINT ligne_i;
    INSERT INTO Ligne_reservation (...) VALUES (...);

    -- 1) Pass : réservation atomique dans le quota (un seul ordre SQL)
    UPDATE Festival SET nbPassVendus = nbPassVendus + :nbPass
     WHERE idFestival = :f AND nbPassVendus + :nbPass <= quotaPass;
    SI 0 ligne modifiée → ROLLBACK TO ligne_i ; sélection retirée ; CONTINUER
    INSERT INTO Pass (idPass, codePass, idFestival, idLigneReservation) ...  -- nbPass fois

    -- 2) Nuitées : prendre uniquement celles encore libres
    UPDATE Nuitee SET idLigneReservation = :ligne
     WHERE idEtablissement = :e AND numHebergement = :h
       AND dateNuitee BETWEEN :debut AND :fin
       AND idLigneReservation IS NULL;
    SI lignes modifiées < nuits demandées → ROLLBACK TO ligne_i ; sélection retirée
  FIN POUR

  SI aucune ligne servie → ROLLBACK ; remboursement total
  SINON → ajuster le montant ; remboursement partiel éventuel
COMMIT;
```

- **Pourquoi un UPDATE conditionnel** plutôt que « SELECT puis UPDATE » : la vérification et l'écriture sont atomiques. Il n'existe aucune fenêtre entre « je lis qu'il reste une place » et « je la prends ».
- **Côté JPA :** ces deux ordres s'écrivent en requêtes `@Modifying`, en testant le nombre de lignes modifiées. On peut aussi utiliser `@Version` (verrou optimiste) sur Festival, avec quelques nouvelles tentatives en cas de conflit.
- **Index nécessaires :** la clé (idEtablissement, numHebergement, dateNuitee) sert directement à l'UPDATE des nuitées. Il faut aussi un index sur `Pass.idFestival`.

Variante « paquets » d'hébergement (évoquée à l'audit)

Si l'on gère un stock (« 40 chambres de 2 places ») au lieu de chambres numérotées, la nuitée devient un compteur `nbRestant` par (hébergement, date). On réserve avec `UPDATE … SET nbRestant = nbRestant - :n WHERE … AND nbRestant >= :n`, sur le même principe que le quota de Pass. C'est plus simple pour l'hébergeur, mais cela crée un point chaud par paquet. La version chambre par chambre répartit mieux les verrous.

6. Passage à l'échelle

Levier	Effet
Panier hors base, transaction seulement à la validation	Les transactions durent quelques millisecondes et aucun verrou n'est tenu pendant que l'utilisateur réfléchit.
Verrous de ligne (Oracle)	Deux réservations sur des hébergements différents ne se gênent jamais.
Compteur `nbPassVendus` et UPDATE conditionnel	Le point chaud par festival est réduit à un seul ordre SQL très court.
Ordre de verrouillage fixe	Pas d'interblocage, donc pas de transactions annulées par Oracle.
Nouvelle tentative avec délai en cas de conflit	Absorbe les pics sans erreur visible pour l'utilisateur.
Pool de connexions et serveur réactif	Beaucoup de requêtes simultanées avec peu de threads.

7. Comment le démontrer

- Un test qui lance N validations simultanées sur un festival dont le quota restant est 1 : **une seule** doit réussir.
- Le même test sur une nuitée libre : une seule ligne obtient la nuitée, les autres voient leur sélection retirée.
- Une démonstration avec deux sessions SQL (SQL Developer) montrant le blocage puis la libération du verrou.

8. Ce que cela implique dans le MCD

- Ajouter `nbPassVendus` dans Festival, ou à défaut compter les Pass par festival sous verrou.
- Garder Occupation en 0,1 côté Nuitee et identifier Nuitee par (hébergement, date).
- Rattacher Pass au Festival (voir le document des modifications).
- Garder `statutPaiement` avec la valeur REMBOURSE.